

AIOnSite — Agent / LLM / Orchestration Backend

Complete Implementation & Integration Documentation

Project	SIH26117 — Sovereign On-Premise Agentic AI Workbench
Scope of this document	Person 1 deliverable: agent orchestration backend built on the Phase-1 skeleton
Branch	feature/agent-orchestration-backend
Pull request	whysist/AIOnSite PR #2 → main
Commit	743677e
Date	2026-09-04
Test status	84 passed (pytest -q) — no network, no API keys required
Python	3.11+
Audience	The 6-person team — especially P2 (RAG/KB), P3 (document intelligence), P4 (tools/DB), P5 (verification/governance), P6 (UI/HITL)

Read this if you are P2–P6: jump to §20 Integration guide (per person). It tells you exactly which function/class the orchestrator will call for your component, the data shape it expects back, and where to plug in. The rest of the document is the reference behind those contracts.

Table of contents

1. Executive summary
2. What changed vs. the Phase-1 skeleton
3. Quick start (run it in 5 minutes)
4. Architecture & end-to-end flow
5. Repository / module map
6. Configuration layer (src/core/config.py)
7. Logging (src/core/logging.py)
8. Exceptions (src/core/exceptions.py)
9. LLM abstraction (src/llm/)
10. Providers (OpenAI-compatible, Ollama, vLLM, local, echo)
11. Agents (src/agents/base_agent.py, roles.py, inspection_agent.py)
12. Planner (src/agents/planner.py)
13. Pipeline models (src/pipeline/models.py)
14. Pipeline graph / DAG (src/pipeline/graph.py)
15. Pipeline builder (src/pipeline/builder.py)
16. Pipeline execution engine (src/pipeline/executor.py)
17. Execution state (src/pipeline/state.py)
18. Model / agent router (src/agents/router.py)
19. Tool system (src/tools/)
20. Verification (src/verification/)
21. Memory (src/memory/)
22. Audit (src/audit/)
23. Orchestrator (src/orchestrator.py)

24. HTTP API (src/api/)

25. CLI entry point (src/main.py) & demo script

26. Security & sovereignty model

27. Testing

28. Integration guide (per person P2–P6)

29. Known limitations / not yet implemented

30. Extension recipes

31. Appendix — dependency changes, full file inventory, glossary

1. Executive summary

The Phase-1 commit contained a **folder skeleton**: abstract base classes with `raise NotImplementedError`, a one-field `Settings`, providers that returned raw OpenAI-SDK objects, agents that returned their input unchanged, and four `assert True` tests.

This work turns that skeleton into a **working backend** for the agentic workbench, implementing the full path:

```
task → plan → pipeline DAG → per-node model routing → agent + tool execution
      → verification → final answer → complete audit trail
```

Design principles held throughout:

- **Provider independence.** No agent, planner, verifier, or pipeline code imports a model vendor. Everything goes through `BaseLLM`. Swapping Ollama → vLLM → OpenAI is a **config change**, never a code change.
- **Local-first / sovereign.** A `SOVEREIGN_MODE` flag (and a per-task `confidential` flag) makes cloud providers impossible to select — enforced at config load *and* again at routing time, and recorded in the audit trail.
- **Never trust LLM JSON.** Plans and verification verdicts are parsed, validated with `Pydantic` + graph checks, retried on failure, and fall back to a deterministic plan if the model cannot comply.
- **Bounded everything.** Retries, replans, tool-loop iterations, and per-node timeouts are all capped. There are no unbounded loops.
- **Observable.** Every execution produces an ordered, machine-readable audit trail and a JSON pipeline snapshot rich enough for a frontend to render without scraping logs.
- **Backward compatible.** Every pre-existing public name still works (`settings`, `BaseAgent.run(state)`, `MemoryManager`, `create_llm`, `OpenAIProvider`, `LocalProvider`, `AgentRouter`, `AgentState`, `LLMResult`).
- **Runs offline.** A deterministic `echo` provider lets the entire pipeline, the demo, and the whole test suite run on an air-gapped machine with no model server.

Numbers: 81 files changed, ~5,400 insertions; 40 new source modules; 84 tests (0 network, 0 keys); `ruff` clean.

2. What changed vs. the Phase-1 skeleton

Area	Phase-1 skeleton	Now
core/config.py	6 plain fields, no validation	Pydantic Settings, enums, validators, <code>get_settings()</code> cache, sovereign enforcement, <code>safe_dump()</code>
core/logging.py	basicConfig + JSON renderer	structlog, console/JSON auto, <code>bind_execution(execution_id=...)</code> context
core/exceptions.py	4 classes	12-class hierarchy, machine codes, <code>to_dict()</code>
llm/base.py	<code>generate()</code> → <code>NotImplementedError</code>	<code>generate</code> / <code>complete</code> / <code>generate_json</code> , normalisation, JSON extraction, error wrapping

Area	Phase-1 skeleton	Now
llm/schemas.py	2 thin models	Role, Message, LLMRequest, UsageMetadata, LLMResponse (+ LLMResult alias)
llm providers	returned raw OpenAI SDK objects	httpx-based, normalised; new : Ollama (native API), vLLM, echo
llm/factory.py	create_llm(provider, **kw), openai/local only	create_llm(settings, provider=..., model=...), 5 providers, sovereign refusal
agents/base_agent.py	BaseAgent.run(state) abstract	+ Agent / LLMAgent with injected model, bounded tool-use loop
agents/	AgentRouter dict lookup	+ roles.py (role agents), planner.py, ModelRouter policy
pipeline/	did not exist	models.py, graph.py, builder.py, executor.py, state.py
tools/	BaseTool, ToolRegistry (no schemas)	typed I/O schemas, permissions, per-call logging, permission-gated registry, 5 built-in tools
verification/	README.md stub	schemas.py, verifier.py (rule-based + optional LLM critique)
memory/	sync MemoryManager	+ MemoryStore ABC, InMemoryStore (TTL); manager kept as wrapper
audit/	README.md stub	events.py (16 event types), trail.py (append-only, redacting, pluggable sink)
api/app.py	/health only	/health, POST /tasks, GET /tasks/{id}, /pipeline, /audit, GET /tasks
orchestration	none	src/orchestrator.py wires the whole flow
main.py	prints 3 lines	--serve / --task / --confidential, provider health check
scripts/run_demo.py	called old main	full offline end-to-end demo with formatted output
tests	4 × assert True	84 real tests across 8 packages + conftest.py
project	—	.env.example, pyproject.toml, trimmed requirements.txt, expanded .gitignore, rewritten README.md, updated configs/*.yaml

Nothing was deleted except the four placeholder tests (replaced by real ones) and the one-time bootstrap script reference (kept in .gitignore). The RAG / OCR / vision directories (src/retrieval/, src/vision/, src/database/) are **untouched** — they keep their README stubs and belong to P2/P3/P4.

3. Quick start (run it in 5 minutes)

```
# 1. environment
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # POSIX
pip install -r requirements.txt

# 2. config
cp .env.example .env # defaults are fine for the offline demo

# 3. run the full pipeline offline (no model server needed)
python scripts/run_demo.py

# 4. run the test suite
pytest -q # -> 84 passed

# 5. start the HTTP API
python -m src.main --serve --port 8080
curl -s localhost:8080/health
```

AGENTS — Agents/LLMs/Orchestration Backend | commit 745077e | page 3 of 20

To use a **real local model**:

```
# install Ollama from https://ollama.com
ollama serve
ollama pull qwen2.5:7b-instruct

# .env
LLM_PROVIDER=ollama
LLM_MODEL=qwen2.5:7b-instruct
OLLAMA_BASE_URL=http://localhost:11434
SOVEREIGN_MODE=true

python scripts/run_demo.py # same demo, now against the real model
python -m src.main --task "Analyze V-101: compare observed pressure 120 bar vs approved limit 100 bar and recommend."
```

4. Architecture & end-to-end flow



Provider abstraction (the rule that must never be broken):

```
Settings ■■ create_llm(settings, provider=?, model=?) ■■ BaseLLM
    ■■ OpenAICompatibleProvider (openai)
    ■■ OllamaProvider (ollama)
    ■■ VLLMProvider (vllm)
    ■■ LocalProvider (local)
    ■■ EchoProvider (echo, offline)
```

Agents receive a BaseLLM. They do not know or care which provider it is.

5. Repository / module map

6. Configuration layer (`src/core/config.py`)

Enums

```
class Environment(str, Enum):    DEVELOPMENT | TESTING | PRODUCTION
class LLMProvider(str, Enum):    OPENAI | LOCAL | OLLAMA | VLLM | ECHO
#    .is_cloud -> True only for OPENAI
```

`_LOCAL_PROVIDERS = {LOCAL, OLLAMA, VLLM, ECHO}` — the set allowed under sovereign mode.

Settings fields

Field	Type	Default	Notes / validation
<code>environment</code>	<code>Environment</code>	<code>development</code>	enum-validated
<code>log_level</code>	<code>str</code>	<code>INFO</code>	must be a valid logging level (upper-cased)
<code>configs_dir</code>	<code>Path</code>	<code>configs</code>	where <code>models.yaml</code> is read from
<code>llm_provider</code>	<code>LLMProvider</code>	<code>openai</code>	enum-validated
<code>openai_api_key</code>	<code>str</code> \ <code>None</code>	<code>None</code>	secret — redacted by <code>safe_dump()</code>
<code>openai_base_url</code>	<code>str</code> \ <code>None</code>	<code>None</code>	
<code>local_llm_base_url</code>	<code>str</code> \ <code>None</code>	<code>http://localhost:8001/v1</code>	
<code>local_llm_api_key</code>	<code>str</code> \ <code>None</code>	<code>local</code>	secret
<code>ollama_base_url</code>	<code>str</code>	<code>http://localhost:11434</code>	
<code>vllm_base_url</code>	<code>str</code> \ <code>None</code>	<code>http://localhost:8000/v1</code>	
<code>vllm_api_key</code>	<code>str</code> \ <code>None</code>	<code>local</code>	secret
<code>llm_model</code>	<code>str</code> \ <code>None</code>	<code>None</code>	provider default if unset
<code>llm_temperature</code>	<code>float</code>	<code>0.2</code>	$0.0 \leq x \leq 2.0$
<code>llm_max_tokens</code>	<code>int</code>	<code>2000</code>	$0 < x \leq 200_000$
<code>request_timeout_seconds</code>	<code>float</code>	<code>60.0</code>	> 0
<code>max_retries</code>	<code>int</code>	<code>2</code>	$0 \leq x \leq 10$ — per pipeline node
<code>max_replans</code>	<code>int</code>	<code>1</code>	$0 \leq x \leq 5$ — planner attempts = <code>max_replans + 1</code>
<code>node_timeout_seconds</code>	<code>float</code>	<code>120.0</code>	> 0 — default per-node timeout
<code>sovereign_mode</code>	<code>bool</code>	<code>False</code>	see below

Cross-field validation

- `log_level` must be one of `CRITICAL/ERROR/WARNING/INFO/DEBUG/NOTSET` (case-insensitive; stored upper-case).
- Sovereign guard (`model_validator`):** if `sovereign_mode` is `True` and `llm_provider` is not in `_LOCAL_PROVIDERS`, construction raises `ValidationError`. So `SOVEREIGN_MODE=true + LLM_PROVIDER=openai` in `.env` **fails fast at startup**.

API

Env vars

Field names map 1:1 to upper-snake env vars (llm_provider → LLM_PROVIDER). Precedence: real env var → .env → default. .env is git-ignored; .env.example is the template.

configs/*.yaml

Non-secret only. models.yaml is the one the runtime reads (via the orchestrator → ModelRouter):

```
default: { provider: ollama, model: qwen2.5:7b-instruct, temperature: 0.2 }
openai:  { model: gpt-4.1-mini }
ollama:  { model: qwen2.5:7b-instruct }
vllm:    { model: qwen2.5:7b-instruct }
local:   { model: local-model }
routing: # deterministic routing by a "complexity" tag on a node
  simple: { model: qwen2.5:7b-instruct }
  complex: { model: qwen2.5:7b-instruct }
  confidential: { provider: ollama, model: qwen2.5:7b-instruct }
```

settings.yaml and tools.yaml are currently documentation-level (they describe intended policy). tools.yaml lists the built-in tool allow-list and permission allow-list; wiring it into default_registry() is a small, isolated follow-up.

7. Logging (src/core/logging.py)

Built on structlog over the stdlib.

```
configure_logging(level="INFO", json_output=None)
# json_output=None -> JSON when stdout is not a TTY, console otherwise
get_logger(name=None) -> structlog BoundLogger # lazily configures if needed
bind_execution(**kv)    # e.g. bind_execution(execution_id="exec_ab12")
clear_execution()        # drop bound context
```

Every log line: ISO-8601 UTC timestamp, level, module, plus any context bound via bind_execution. The orchestrator binds execution_id for the duration of a run, so every log line from planner/executor/agents/tools/verifier carries the execution id. Secrets are never rendered — callers must pass already-safe values (helpers safe_dump() / audit redaction do this).

8. Exceptions (src/core/exceptions.py)

All inherit AIOnSiteError(message, *, details: dict | None), which has:

```
.message : str
.details : dict
.code    : str # machine-readable, surfaced in API error payloads
.to_dict() -> {"error": code, "message": message, "details": details}
```

Class	code	Raised when
AIOnSiteError	aionsite_error	base
ConfigurationError	configuration_error	invalid/missing config (get_settings() wraps ValidationError)
LLMError	llm_error	generic LLM-layer failure; BaseLLM.complete wraps unexpected provider errors
ProviderError(LLMError)	provider_error	provider unreachable / HTTP ≥ 400 / bad response shape
SovereigntyError	sovereignty_error	a cloud route was attempted under sovereign/confidential policy
AgentExecutionError	agent_execution_error	an agent failed (e.g. no LLM bound)

Class	Code	Raised when
PlanningError (AgentExecutionError)	planning_error	reserved for planner failures the API should surface
PipelineExecutionError	pipeline_execution_error	cycle detected, or all branches failed with no answer
ToolExecutionError	tool_execution_error	tool raised or returned invalid output
ToolNotFoundError (ToolExecutionError)	tool_not_found	registry lookup miss
VerificationError	verification_error	verifier could not evaluate

The API catches `AIOnSiteError` centrally and returns `ErrorResponse` (details hidden in production).

9. LLM abstraction (src/llm/)

Schemas (schemas.py)

```
class Role(str, Enum): SYSTEM | USER | ASSISTANT | TOOL

class Message(BaseModel):
    role: Role
    content: str
    name: str | None = None
    def as_dict() -> {"role": ..., "content": ..., ["name": ...]}

class LLMRequest(BaseModel):
    messages: list[Message]
    model: str | None = None
    temperature: float = 0.2 # 0..2
    max_tokens: int = 2000 # > 0
    stop: list[str] | None = None
    json_mode: bool = False # ask provider for JSON (best-effort)
    metadata: dict = {} # e.g. {"aionsite_kind": "plan"} – see echo provider
    @classmethod from_messages(messages: list[dict|Message], **kw) -> LLMRequest
    wire_messages() -> list[dict] # provider wire format

class UsageMetadata(BaseModel):
    prompt_tokens / completion_tokens / total_tokens : int # supports +

class LLMResponse(BaseModel):
    content: str
    model / provider / finish_reason : str | None
    usage: UsageMetadata
    latency_ms: float | None
    created_at: float
    raw: dict | None # provider payload, repr-hidden
    metadata: dict

LLMResult = LLMResponse # backward-compatible alias
```

BaseLLM (base.py)

```
class BaseLLM(ABC):
    provider_name: str = "base"
    def __init__(self, *, model=None, is_local=True)

    async def generate(messages: list[dict|Message], **kw) -> LLMResponse
        # kw: model, temperature, max_tokens, stop, json_mode, metadata
    async def complete(request: LLMRequest) -> LLMResponse
        # fills model from self.model, times the call, wraps errors in LLMError,
        # sets latency_ms / provider if provider left them unset
    async def generate_json(messages, **kw) -> Any
        # json_mode=True by default; parses with extract_json()

    @staticmethod extract_json(text) -> Any
        # tolerant: strips ``` fences, falls back to first {...}/[...] block;
        # raises LLMError if nothing parseable
    @staticmethod system(text) -> Message
    @staticmethod user(text) -> Message

    @abstractmethod async def _complete(request: LLMRequest) -> LLMResponse # subclasses implement this
    async def health_check() -> bool # network providers override
    async def aclose() -> None # closes httpx clients
```

Contract: subclasses implement only `_complete`, translating their native output into an `LLMResponse`. Nothing outside `src/llm/` ever sees a provider-native object.

Factory (`factory.py`)

```
create_llm(settings: Settings | None = None, *, provider=None, model=None) -> BaseLLM
```

- settings defaults to `get_settings()`.
- `provider` / `model` override the config (the router uses this per node).
- If `sovereign_mode` and the resolved provider is `openai` → raises `SovereigntyError`.
- Maps the enum/string to a class; unknown → `ConfigurationError`.

10. Providers

All providers are **httpx-based**. The `openai` Python SDK is **not a dependency** of this backend (kept as an optional comment in `requirements.txt` for other workstreams).

Provider	Class	provider_name	is_local	Endpoint	Wire API
OpenAI-compatible	<code>OpenAICompatible</code> (alias <code>OpenAIProvider</code>)	<code>propenai</code>	<code>False</code>	<code>openai_base_url</code> or <code>https://api.openai.com/v1</code>	<code>POST /chat/completions</code>
Ollama	<code>OllamaProvider</code>	<code>ollama</code>	<code>True</code>	<code>ollama_base_url (:11434)</code>	<code>POST /api/chat (native)</code>
vLLM	<code>VLLMProvider</code>	<code>vllm</code>	<code>True</code>	<code>vllm_base_url (:8000/v1)</code>	<code>POST /chat/completions</code>
Generic local	<code>LocalProvider</code>	<code>local</code>	<code>True</code>	<code>local_llm_base_url (:8001/v1)</code>	<code>POST /chat/completions</code>
Echo (offline)	<code>EchoProvider</code>	<code>echo</code>	<code>True</code>	<code>none</code>	<code>none</code>

Common behaviour:

- **Errors** → `ProviderError` with a helpful message. Ollama specifically:
 - connection refused → `"Cannot reach Ollama at ... Is it running? Try: ollama serve and ollama pull <model>."`
 - HTTP 404 → `"ollama model '<m>' is not installed. Run: ollama pull <m>"`
 - `json_mode=True`:
 - OpenAI-compatible → sends `response_format={"type":"json_object"}`, retries once without it on HTTP 400.
 - Ollama → sends `"format":"json"`.
 - `health_check()` — OpenAI-compatible: `GET /models (< 500)`; Ollama: `GET /api/tags (== 200)`.
- `OllamaProvider.list_models()` returns installed model names.
- `aclose()` closes the `httpx AsyncClient`.

EchoProvider — the offline engine

Deterministic, **no network**. It is *not* a language model; it returns rule-based output shaped so the pipeline can parse it. It branches on `request.metadata["aionsite_kind"]` (real providers ignore this key):

aionsite_kind	Output
<code>"plan"</code>	a valid 3-step JSON plan (researcher → analyst → summarizer) whose goal is the first line of the task
<code>"verify"</code>	<code>{"passed": true, "score": 0.8, "issues": [], "recommendation": "accept", "rationale": "..."} </code>

aionsite_king	Output
anything else	a labelled [echo:<role>] text summary of the input, plus a note to switch to a real provider

This is why `pytest`, `scripts/run_demo.py`, and `python -m src.main` all work with zero setup.

11. Agents (src/agents/)

BaseAgent (legacy, kept)

```
class BaseAgent(ABC):
    @abstractmethod async def run(self, state) -> state
```

Used by `InspectionAgent` and legacy tests. Not used by the pipeline.

Agent (new base)

```
class Agent:
    def __init__(self, *, name, description="", system_prompt="", llm: BaseLLM | None = None,
                  tools: list[str] | None = None, temperature=0.2, max_tokens=2000,
                  max_tool_iterations=3, metadata: dict | None = None):
    def with_llm(self, llm) -> Agent # shallow copy bound to a model (router uses this)
    async def execute(self, task: str, *, context: dict | None = None,
                     registry: ToolRegistry | None = None) -> AgentResult
```

The model is **injected**, never constructed inside the agent.

LLMAgent (Agent) — the concrete pipeline agent

`execute()`:

- Builds messages: `system_prompt` (if any) + one user message = `TASK:\n<task> + CONTEXT FROM PREVIOUS STEPS:` (each dependency node's output, labelled `[node_id]`).
- If the node has tools that exist in the registry, appends tool instructions:

```
"To call a tool, reply with ONLY a JSON object: {"tool": "<name>", "arguments": {...}}. After the tool result you may call another tool or give your final answer as plain text. Available tools: ..."
```

- Bounded tool-use loop** (`max_tool_iterations`, default 3): - call the model; - if the reply parses as `{"tool": ..., "arguments": {...}}` and the tool exists → run it via the registry, append an assistant message + a `role="tool"` message with `{"ok", "output", "error"}`, loop; - otherwise → break.
- Returns `AgentResult` with output (final text), `tool_results`, `model`, `provider`, token counts, `duration_ms`, `metadata={"tool_calls": n}`.

With the `echo` provider the model never emits a tool call, so `execute()` returns after one round — the loop is real but only engages with capable models.

Role library (roles.py)

```
build_agent_library(llm: BaseLLM) -> dict[str, LLMAgent]
```

Returns agents keyed by role, each with a role-specific system prompt and a default tool set:

Role	Purpose	Default tools
researcher	gather & organise facts; never invent evidence	read_file, text_stats, json_parse
analyst	reason & compute; prefer tools for arithmetic	calculator, calculate_deviation, json_parse
executor	perform a concrete step / tool action	all five built-ins

Role	Purpose	Default tools
summarizer	synthesise the final answer from prior outputs	—
custom	generic fallback	—

Every `NodeType` value is also registered as a key (aliased to `custom` where no dedicated role exists) so planner output that names a node-type still resolves.

InspectionAgent(BaseAgent)

Domain entry agent. With an LLM bound it runs an `LLMAgent` over the task in `state` and writes the answer + full `AgentResult` into `state.metadata["inspection"]`. With **no** LLM it records `{"status": "no_model"}` — it never fabricates an answer.

12. Planner (src/agents/planner.py)

Turns a natural-language task into a **validated** structured plan.

Models

```
class PlanStep(BaseModel):
    id: str # non-empty (validator strips + rejects blank)
    agent: str = "custom" # validator lower-cases; unknown -> "custom" (logged)
    description: str = ""
    depends_on: list[str] = []
    tools: list[str] = []

class Plan(BaseModel):
    goal: str
    steps: list[PlanStep] # validator enforces all of:
    # - non-empty
    # - unique ids
    # - no self-dependency
    # - every depends_on target exists
    # - every depends_on target is defined *earlier* (⇒ acyclic by construction)
```

Known agent names: `researcher`, `analyst`, `executor`, `summarizer`, `verifier`, `custom`.

Planner

```
Planner(llm: BaseLLM, *, max_attempts: int = 2)
async def plan(self, task: str, *, context: dict | None = None) -> Plan
@staticmethod fallback_plan(task: str) -> Plan
```

Loop (`max_attempts`, orchestrator sets it to `MAX_REPLANS + 1`):

1. Ask the model (system prompt fixes the JSON schema, `temperature=0.0`, `metadata={"aionsite_kind": "plan"}`).
2. `extract_json` → `Plan.model_validate`.
3. On any failure: log it, append *"Your previous response was invalid: . Return corrected JSON only."* and retry.
4. After the budget is exhausted → `fallback_plan(task)` — a deterministic `researcher` → `analyst` → `summarizer` chain.

The pipeline can therefore always run.

Planner JSON contract (what the model is asked to emit)

```
{
  "goal": "<one sentence>",
  "steps": [
    {"id": "step_1", "agent": "researcher", "description": "...", "depends_on": []},
    {"id": "step_2", "agent": "analyst", "description": "...", "depends_on": ["step_1"]},
    {"id": "step_3", "agent": "summarizer", "description": "...", "depends_on": ["step_2"]}
  ]
}
```

Rules enforced: 2–6 steps, unique ids, `depends_on` references earlier ids only, end with a summarizer step.

13. Pipeline models (src/pipeline/models.py)

Pure data — no imports of agents/providers, so every layer can depend on it.

Enums

```
NodeType:          planner | researcher | analyst | executor | verifier | summarizer | custom
NodeStatus:        pending | running | completed | failed | skipped | retrying | cancelled
ExecutionStatus:    pending | running | completed | failed | needs_review | cancelled
```

RetryPolicy

```
max_retries: int = 1          # 0..10
backoff_seconds: float = 0.5
delay_for(attempt) -> backoff_seconds * max(1, attempt)
```

ToolResult

```
tool: str
call_id: str          # "tool_XXXXXXXXXX"
ok: bool = True
input: dict           # sanitised (secrets redacted, long strings truncated)
output: Any
error: str | None
duration_ms: float | None
started_at: float
```

AgentResult

```
agent: str
node_id: str | None
run_id: str          # "run_XXXXXXXXXX"
ok: bool
output: str          # the agent's text output
structured: dict | None # e.g. a verifier verdict
tool_results: list[ToolResult]
model / provider: str | None
prompt_tokens / completion_tokens: int
duration_ms: float | None
error: str | None
metadata: dict
```

PipelineNode

```
id: str              # "node_xxxx" or the planner step id
name: str
type: NodeType = custom
description: str
agent: str           # role key resolved against the agent library
inputs / outputs: list[str]
depends_on: list[str]
tools: list[str]
provider / model: str | None # per-node override honoured by the router; filled by executor with the decision
require_local: bool = False # forces local routing for this node
retry_policy: RetryPolicy
timeout_seconds: float | None
# runtime fields (mutated during execution):
status: NodeStatus = pending
attempts: int = 0
started_at / finished_at: float | None
result: AgentResult | None
error: str | None
.duration_ms -> (finished_at - started_at) * 1000
```

PipelineEdge

```
source: str
target: str
condition: str | None # modelled for conditional edges; predicate evaluation is not yet implemented
```

Pipeline

```
id: str                # "pipeline_xxxx"
goal: str
nodes: list[PipelineNode]
edges: list[PipelineEdge]
metadata: dict
# model_validator enforces: unique node ids; every depends_on / edge endpoint exists; no self-dependency
.node(node_id) -> PipelineNode
.as_graph_dict() -> dict    # the frontend contract – see §24
```

14. Pipeline graph / DAG ([src/pipeline/graph.py](#))

```
class PipelineGraph:
    def __init__(self, pipeline: Pipeline)    # builds dep / dependent maps from depends_on AND edges
    dependencies(node_id) -> set[str]
    dependents(node_id) -> set[str]
    detect_cycle() -> list[str] | None        # returns the offending cycle, or None
    topological_order() -> list[str]         # raises PipelineExecutionError on a cycle
    execution_layers() -> list[list[str]]    # layer k = nodes whose deps are all in layers 0..k-1
    validate() -> None                       # raises PipelineExecutionError if a cycle exists
```

`execution_layers()` is what the executor uses: each layer is a set of nodes with no unmet dependency, run concurrently.

15. Pipeline builder ([src/pipeline/builder.py](#))

```
build_pipeline(plan: Plan, *, settings: Settings | None = None,
               add_verifier: bool = True, require_local: bool = False) -> Pipeline
```

- Each `PlanStep` → a `PipelineNode` with **the same id** (traceability: one id from planner → pipeline → audit → frontend).
- `NodeType` inferred from the step's agent.
- `depends_on` copied; one `PipelineEdge` per dependency.
- `require_local = require_local` or `settings.sovereign_mode` — stamped on every node.
- `retry_policy = RetryPolicy(max_retries=settings.max_retries)`, `timeout_seconds = settings.node_timeout_seconds`.
- If `add_verifier` (default): appends a terminal node `id="verify_final"`, `type=VERIFIER`, `agent="verifier"`, depending on **all leaf nodes**, with `max_retries=0`.
- Runs `PipelineGraph(pipeline).validate()` before returning.

16. Pipeline execution engine ([src/pipeline/executor.py](#))

```
class PipelineExecutor:
    def __init__(self, *, agent_library: dict[str, Agent], registry: ToolRegistry,
                 router: ModelRouter, audit: AuditTrail, verifier: ResultVerifier,
                 settings: Settings | None = None)
    async def run(self, ctx: ExecutionContext, pipeline: Pipeline) -> ExecutionContext
```

`run()`

1. `bind_execution(execution_id=ctx.execution_id)`.
2. `layers = PipelineGraph(pipeline).execution_layers()`; audit `PIPELINE_BUILT` with the layer list.
3. For each layer: `await asyncio.gather(*(_run_node(...) for nid in layer))` — **parallel within a layer, sequential across layers**.
4. `_finalise(ctx, pipeline)`.

`_run_node()`

- **Dependency gate:** if any `depends_on` node is `FAILED` / `SKIPPED` / `CANCELLED` → this node becomes `SKIPPED`, audit `NODE_SKIPPED`, return (its own dependents will then also skip).
- `status = RUNNING`, `started_at`, audit `NODE_STARTED`.
- `context = ctx.state.outputs_for(node.depends_on) — {dep_id: dep_output_text}`.
- **Verifier node** → `_run_verifier_node()` (below).
- Otherwise: resolve `agent = agent_library[node.agent]` (fallback `"custom"`); if none → `FAILED` + `NODE_FAILED`.
- `llm, decision = router.get_llm(node)`; set `node.provider`, `node.model` from the decision; `bound = agent.with_llm(llm)`.
- **Retry loop** (`1 .. max_retries + 1`):
- `result = await asyncio.wait_for(bound.execute(task, context=context, registry=registry), timeout=node.timeout_seconds or settings.node_timeout_seconds)`
- `task text = "<node.description>\n\n(Overall goal: <pipeline.goal>)"`.
- **success** → `node.result, status=COMPLETED, finished_at, ctx.state.record(...)`; audit one `TOOL_CALLED/TOOL_FAILED` per tool call, then `NODE_COMPLETED` (with `provider/model/duration/tool` count). Return.
- **failure** (timeout or exception): if attempts remain → `status=RETRYING`, audit `RETRY_TRIGGERED`, `await asyncio.sleep(retry_policy.delay_for(attempt))`, continue; else break.
- After the loop with no success → `status=FAILED`, `node.error`, append `{"node", "error"}` to `ctx.state.errors`, audit `NODE_FAILED`.

`_run_verifier_node()`

- `combined = "\n\n".join(f"[{dep}]\n{output}" ...)` over dependency outputs.
- `verdict = await asyncio.wait_for(verifier.verify(ctx.task, combined, require_evidence=False), timeout=...)`; on exception → a `VerificationResult(passed=False, recommendation=ESCALATE, checker="error")`.
- Stores `ctx.state.verifications[node.id] = verdict`; builds an `AgentResult(agent="verifier", structured=verdict.model_dump())`; `status=COMPLETED`.
- Audit `VERIFICATION_COMPLETED` (status ok/failed, score, recommendation, issues).

`_finalise()`

- **Final answer:** last `COMPLETED` node that is a `SUMMARIZER`; if none, the last `COMPLETED` non-verifier node; take its `result.output`.
- **Final verification:** the last `VERIFIER` node's verdict, if present.
- If some nodes `FAILED` and there is no final answer → raise `PipelineExecutionError("all pipeline branches failed", details={"failed_nodes": [...]}).`

17. Execution state (`src/pipeline/state.py`)

No global mutable state — one `ExecutionContext` per run, threaded everywhere.

`PipelineState`

```
node_results: dict[node_id, AgentResult]
tool_results: list[ToolResult]           # flattened across nodes
verifications: dict[node_id, VerificationResult]
errors: list[dict]                       # [{"node", "error"}, ...]
scratch: dict

record(node_id, result)                  # store result + extend tool_results
outputs_for(node_ids) -> {node_id: result.output}
```

`ExecutionContext`

18. Model / agent router (`src/agents/router.py`)

AgentRouter (legacy, kept)

```
AgentRouter(agents: dict).get_agent(name) -> agent | None
```

ModelRouter

```
ModelRouter(settings: Settings | None = None, *,
             model_config: dict | None = None, # parsed configs/models.yaml
             confidential: bool = False)

route(node: PipelineNode) -> RouteDecision
get_llm(node: PipelineNode) -> tuple[BaseLLM, RouteDecision] # cached by (provider, model)
async aclose() # closes all cached llms
```

RouteDecision

```
provider: str
model: str | None
reason: str # "policy: local-only" | "node override" | "configured default"
is_local: bool
.as_dict()
```

Policy (in `route()`)

1. `local_only` = `sovereign_mode` or `confidential` or `node.require_local`.
2. `requested` = `node.provider` or `settings.llm_provider`.
3. If `local_only` and `requested` is a cloud provider: - if `settings.provider_allowed(requested)` is `False` → **raise `SovereigntyError`** (never silently downgrade under a hard policy); - else pick the first local provider (`ollama` → `local` → `vllm` → `echo`) and log `route_forced_local`.
4. `Model`: `node.model` → `models.yaml routing[<complexity>].model` → `models.yaml <provider>.model` → `settings.llm_model`.
5. `get_llm()` calls `create_llm(settings, provider=decision.provider, model=decision.model)` and caches the instance.

19. Tool system (`src/tools/`)

BaseTool (`base_tool.py`)

```
class ToolPermission(str, Enum): PURE | READ_FILESYSTEM("read_fs") | NETWORK | WRITE_FILESYSTEM("write_fs")

class BaseTool(ABC):
    name: ClassVar[str] # unique registry key (required)
    description: ClassVar[str]
    permissions: ClassVar[tuple[ToolPermission, ...]] = (PURE,)
    InputModel: ClassVar[type[BaseModel]] # required
    OutputModel: ClassVar[type[BaseModel] | None] = None

    async def run(self, **kwargs) -> ToolResult
        # 1. validate kwargs against InputModel -> on failure: ToolResult(ok=False, error="invalid arguments: ...")
        # 2. await self._run(args)
        # 3. coerce output via OutputModel if set
        # 4. catch ToolExecutionError / any Exception -> ToolResult(ok=False, error=...)
        # 5. always: duration_ms, structured log line "tool_call" (tool, ok, duration_ms, error)
    @abstractmethod async def _run(self, args: BaseModel) -> Any # subclasses implement this
    @classmethod spec() -> dict # {name, description, permissions, input_schema, output_schema}
```

Argument sanitisation before logging: keys containing key/token/secret/password/authorization → *****redacted*****;
strings > 500 chars → truncated.

ToolRegistry (registry.py)

```
ToolRegistry(*, allowed_permissions: set[ToolPermission] | None = None)
register(tool, *, replace=False)      # raises ToolExecutionError if the tool needs a permission not in allowed_permissions
unregister(name)
get(name) -> BaseTool                 # raises ToolNotFoundError
has(name) -> bool
all() -> list[BaseTool]
names() -> list[str]
specs() -> list[dict]
async call(name, **kwargs) -> ToolResult

default_registry(*, allow_network: bool = False) -> ToolRegistry
# allowed = {PURE, READ_FILESYSTEM} (+ NETWORK if allow_network)
# registers every class in tools.builtin.BUILTIN_TOOLS whose permissions are granted
```

Built-in tools

name	Class	Permission	Input	Output
calculator	CalculatorTool	PURE	{expression: str} (+ - * / // % **, no names/calls) — AST walk, not eval ; exponent range-checked	{expression, result: float}
calculate_deviation	DeviationTool	PURE	{actual: float, limit: float, label?: str}	{actual, limit, absolute_deviation, percent_deviation, within_limit, formula, label}
text_stats	TextStatsTool	PURE	{text: str} (≤ 100k)	{characters, words, sentences, unique_words, avg_word_length}
json_parse	JsonParseTool	PURE	{data: str, path?: str} (dotted path e.g. items.0.value)	{value, type}
read_file	FileReadTool	READ_FILESYSTEM	{relative_path: str, max_bytes?: int ≤ 1_000_000}	{path, bytes, truncated, content}

`FileReadTool(root=...)` defaults `root` to `<cwd>/data`. The resolved path **must** stay inside `root` (`Path.relative_to` to check) — `../escape` → `ToolExecutionError("path escapes the allowed data root")`. Text must be valid UTF-8.

Agent $\leftrightarrow$ tool protocol

The model asks for a tool by emitting exactly:

```
{"tool": "calculate_deviation", "arguments": {"actual": 120, "limit": 100}}
```

The LLMAgent executes it and feeds back a `role="tool"` message:

```
{ "ok": true, "output": { "absolute_deviation": 20.0, "percent_deviation": 20.0, "within_limit": false, "...": "..."}, "error": null }
```

20. Verification (src/verification/)

Schemas (schemas.py)

[illegible]

ResultVerifier (verifier.py)

```
ResultVerifier(llm: BaseLLM | None = None, *, min_score: float = 0.5)
async def verify(self, task: str, result: str, *,
                 require_evidence: bool = False, require_json: bool = False,
                 structured: dict | None = None) -> VerificationResult
```

Layer 1 — rule-based (always runs, no model):

Check	Issue code	Severity
empty output (and no structured)	empty_output	critical
contains traceback... / exception: / error:	error_marker	major
contains however, this contradicts / conflicting / cannot be both	contradiction	major
require_json but not parseable and no structured	malformed_json	major
require_evidence but no source: / page / [/ URL / citation / evidence marker	missing_evidence	major

Score = $1.0 - \sum \text{penalties (minor } 0.1, \text{ major } 0.4, \text{ critical } 1.0)$. passed = no blocking issue and score ≥ 0.5 .
Recommendation: accept if passed; replan if any critical; else retry.

Layer 2 — optional LLM critique (only if an `llm` was supplied):

- Asks the model (temperature=0.0, metadata={"aionsite_kind": "verify"}) for a JSON verdict.
- **Any exception in the critique is swallowed** (logged `llm_critique_failed`) and the rule-based result is returned — a broken/slow critic never fails the run.
- Merge is **conservative**: `score = min(rule, llm)`, `passed = rule.passed and llm.passed` and `score ≥ min_score`, `recommendation = the stricter of the two`.

Never turns "no evidence" into a pass. **Never** loops — retry/replan counts are enforced by the caller.

The pipeline's terminal `verify_final` node calls this with `require_evidence=False` over the concatenated leaf outputs; the orchestrator sets execution status to `needs_review` if the final verdict does not pass.

21. Memory (src/memory/)

```
class MemoryStore(ABC):          # base.py – async
    async get(key) -> Any | None
    async set(key, value, *, ttl_seconds: float | None = None)
    async delete(key)
    async keys(*, prefix="") -> list[str]
    async get_many(keys) -> dict

class InMemoryStore(MemoryStore): # in_memory.py – dict + optional per-key expiry; .clear()

class MemoryManager:             # memory_manager.py – SYNC wrapper (backward compat)
    get/set/delete(session_id[, value]) # raises if called inside a running event loop
    .store -> InMemoryStore
```

The orchestrator does not currently persist to memory between runs; `MemoryStore` is the seam for P2 to add a SQLite / vector-store implementation without touching callers.

22. Audit (src/audit/)

AuditEventType (events.py) — 16 values

AuditEvent (frozen)

```
id: str # uuid hex
timestamp: float
execution_id: str
event_type: AuditEventType
component: str # "orchestrator" | "planner" | "executor" | "tool" | "verifier"
status: str = "ok"
message: str
metadata: dict
```

AuditTrail (trail.py)

```
AuditTrail(on_event: callable | None = None) # on_event(event) -> durable sink (file/DB); exceptions are caught
record(execution_id, event_type, *, component, status="ok", message="", metadata=None) -> AuditEvent
events(execution_id=None) -> list[AuditEvent]
as_dicts(execution_id=None) -> list[dict] # JSON-ready
extend(events)
```

Metadata is deep-redacted before storage: keys matching `key/token/secret/password/authorization/api_key` → `***redacted***`; strings > 2000 chars truncated.

Typical event sequence for one successful run

```
task_received → policy_enforced? → plan_created → pipeline_built
→ (node_started → [tool_called...] → node_completed) × N
→ node_started(verify_final) → verification_completed
→ final_answer_generated → execution_completed
```

GET `/tasks/{id}/audit` returns exactly this list, in order.

23. Orchestrator (src/orchestrator.py)

```
class Orchestrator:
    def __init__(self, settings: Settings | None = None, *,
                  audit: AuditTrail | None = None,
                  registry: ToolRegistry | None = None,
                  use_llm_verifier: bool = True):
        async def run_task(self, task: str, *, confidential: bool = False) -> ExecutionContext
        async def aclose(self) # closes every router's llm clients
```

`run_task()` step by step:

1. Create `ExecutionContext` (`sovereign_mode` from settings, `confidential` from arg). `bind_execution(execution_id=...)`. `status = RUNNING`.
2. Audit `TASK_RECEIVED`. If `sovereign_mode` or `confidential` → audit `POLICY_ENFORCED`.
3. Build a `ModelRouter` (`settings`, `model_config=<models.yaml>`, `confidential=...`).
4. `_run()`: a. Build a synthetic `plan_node` (`require_local = confidential or sovereign_mode`); `planner_llm = router.get_llm(plan_node)`. b. `Planner(planner_llm, max_attempts=MAX_REPLANS + 1).plan(task) → ctx.plan = plan.model_dump()`; audit `PLAN_CREATED` (with all steps). c. `build_pipeline(plan, settings, require_local=...) → ctx.pipeline`. d. `agent_library = build_agent_library(default_llm)`; `verifier = ResultVerifier(default_llm if use_llm_verifier else None)`. e. `PipelineExecutor(...).run(ctx, pipeline)`. f. If `ctx.final_answer` → audit `FINAL_ANSWER_GENERATED`. g. **Status**: no answer → `FAILED` (`error="no final answer produced"`); final verdict not passed → `NEEDS_REVIEW`; some nodes failed → `NEEDS_REVIEW`; else `COMPLETED`. `mark_finished(status)`; audit `EXECUTION_COMPLETED`.
5. Exceptions: `AIOnSiteError` → `ctx.error = exc.message`; anything else → `ctx.error = "<Type>: <msg>"`. Status `FAILED`, audit `EXECUTION_FAILED`. **`run_task` never raises** — the caller inspects `ctx.status / ctx.error`.
6. finally: `clear_execution()`.

24. HTTP API (src/api/)

FastAPI app, title="AIOnSite", version="0.2.0". A lifespan handler builds one shared AppState (Settings, AuditTrail, default_registry(allow_network=False), Orchestrator, and an in-process executions: dict[execution_id, ExecutionContext]), and calls orchestrator.aclose() on shutdown.

Endpoints

Method	Path	Body	Response model	Notes
GET	/health	—	HealthResponse	{status, environment, llm_provider, sovereign_mode, tools[]}
POST	/tasks	TaskRequest	TaskCreatedResponse	runs the task synchronously, stores the context, returns id + status + answer + verification
GET	/tasks	—	list[str]	all known execution ids
GET	/tasks/{execution_id}	—	ExecutionResponse	full result incl. node_results
GET	/tasks/{execution_id}/pipeline	—	PipelineResponse	pipeline = Pipeline.as_graph_dict()
GET	/tasks/{execution_id}	—	AuditResponse	ordered events[]

Unknown execution_id → 404. AIOnSiteError → 400 + ErrorResponse{error, message, details} (details empty in production). Body validation errors → 422 (FastAPI default).

Request / response schemas (api/schemas.py)

TaskRequest	{ task: str (1..20000), confidential: bool = False }
TaskCreatedResponse	{ execution_id, status, final_answer?, verification?: dict, error? }
ExecutionResponse	{ execution_id, status, task, sovereign_mode, confidential, duration_ms?, final_answer?, final_verification?: dict, error?, node_results: { node_id: AgentResult-dict } }
PipelineResponse	{ execution_id, status, pipeline?: <as_graph_dict> }
AuditResponse	{ execution_id, events: [<AuditEvent-dict>] }
HealthResponse	{ status, environment, llm_provider, sovereign_mode, tools: [str] }
ErrorResponse	{ error, message, details: dict }

Pipeline.as_graph_dict() — the frontend contract (P6)

```
{
  "id": "pipeline_...",
  "goal": "...",
  "nodes": [
    {
      "id": "step_1",
      "name": "...",
      "type": "researcher",
      "description": "...",
      "agent": "researcher",
      "depends_on": [],
      "tools": ["read_file", "text_stats", "json_parse"],
      "provider": "ollama",
      "model": "qwen2.5:7b-instruct",
      "status": "completed",
      "attempts": 1,
      "duration_ms": 812.4,
      "error": null,
      "result": { /* full AgentResult.model_dump(), incl. tool_results */ }
    }
    /* ... + a terminal {"id":"verify_final","type":"verifier", ...} node */
  ],
  "edges": [ {"source": "step_1", "target": "step_2", "condition": null} ],
  "metadata": {"source": "planner"}
}
```

```
{
  "execution_id": "exec...", "task": "...", "status": "completed",
  "sovereign_mode": false, "confidential": false,
  "created_at": ..., "finished_at": ..., "duration_ms": ...,
  "replans": 0, "error": null,
  "final_answer": "...",
  "final_verification": { "passed": true, "score": 0.8, "recommendation": "accept", "issues": [], "...": "..." },
  "pipeline": { /* as_graph_dict() */ },
  "node_results": { "step_1": { /* AgentResult */ }, "...": {} },
  "verifications": { "verify_final": { /* VerificationResult */ } }
}
```

25. CLI entry point (src/main.py) & demo script

```
python -m src.main # load settings, configure logging, create the configured
                  # provider, run health_check(), print a safe report
python -m src.main --task "..." # run one task end-to-end, print ExecutionContext.summary() as JSON
python -m src.main --task "..." --confidential
python -m src.main --serve --host 127.0.0.1 --port 8000 # uvicorn src.api.app:app
```

Exit codes: 0 success (or needs_review), 1 failed / provider unreachable, 2 configuration error. Secrets are never printed (Settings.safe_dump()).

scripts/run_demo.py:

- Forces LLM_PROVIDER=echo **only if not already set**, so it is offline by default and honours a real provider when you export one.
- Optional first CLI arg overrides the default V-101 task.
- Prints: TASK → PLAN (steps + deps) → PIPELINE (DAG with per-node status/provider/model) → NODE RESULTS (with tool-call counts) → VERIFICATION → FINAL ANSWER → AUDIT TRAIL → RESULT.

26. Security & sovereignty model

Concern	Mechanism
Secrets	never hard-coded; .env git-ignored; .env.* ignored except .env.example; Settings.safe_dump() redacts; audit + tool-arg logging redact key/token/secret/password/authorization
Cloud egress	LLMProvider.is_cloud (only openai); sovereign guard rejects cloud at config load ; ModelRouter rejects cloud at routing time (SovereigntyError); per-task confidential=true forces local even without sovereign mode; POLICY_ENFORCED audit event records it
Arbitrary code	calculator uses an AST walk (no eval, no names, no calls, exponent range-limited); no shell tool exists
Filesystem	read_file sandboxed to data/; Path.relative_to check blocks ../ and symlink escape; size-capped; UTF-8 only; WRITE_FILESYSTEM permission exists but no tool uses it
Network tools	default_registry(allow_network=False) — NETWORK permission not granted, so no network tool is registered by default
LLM output	plans & verdicts validated with Pydantic + graph checks; malformed → bounded retry → deterministic fallback; verifier never converts "no evidence" into a pass
Loops	max_retries (per node), max_replans (planner), max_tool_iterations (agent loop), node_timeout_seconds — all bounded

Concern	Mechanism
Error leakage	API returns ErrorResponse with details={} in production; full technical detail only in logs
Future	RBAC / auth / policy-engine hooks are architecturally open (no auth middleware yet)

27. Testing

```
pytest -q          # 84 passed, ~5s, no network, no API keys
ruff check src/ tests/
```

tests/conftest.py sets LLM_PROVIDER=echo, ENVIRONMENT=testing, SOVEREIGN_MODE=false, and an autouse fixture calls reload_settings() around every test. Fixtures: settings, echo_llm.

Package	File(s)	Covers
test_core	test_config.py	defaults, provider/env selection, temperature & max-token validation, invalid provider/log-level, sovereign rejects cloud / allows local , safe_dump redaction, get_settings wraps ValidationError
test_llm	test_factory.py	every provider → BaseLLM; unknown provider; sovereign blocks cloud override; is_local flags; model override
	test_base_and_schemas.py	loose-message normalisation, UsageMetadata addition, LLMResponse shape, extract_json (bare / fenced / embedded / garbage), generate_json via echo, provider errors wrapped in LLMError
	test_providers_mocked.py	OpenAI-compatible & Ollama happy paths + HTTP-error → ProviderError, via httpx.MockTransport (no network)
test_agents	test_planner.py	model rejects dup ids / unknown dep / forward dep; unknown agent → custom; parse valid JSON; recover malformed-then-valid; fallback after exhaustion; cyclic plan → fallback
	test_router.py	AgentRouter lookup; default route; node override; sovereign refuses cloud node ; confidential forces local; model from config map
	test_inspection_agent.py	inspection agent with/without LLM; LLMAgent real tool-use loop (model requests calculator, gets 5.0); agent library has core roles
test_pipeline	test_graph.py	topological order; parallel layers; cycle detection; model rejects unknown dependency
	test_builder.py	step ids preserved + verifier appended; sovereign marks nodes local; edges match deps
	test_executor.py	sequential order + context passing; parallel branch → join sees both; retry-then-success; failure propagates → dependents skipped ; verifier node produces a VerificationResult

Package	File(s)	Covers
test_verification	test_verifier.py	pass on clean; empty → replan; error-marker → retry; missing-evidence flagged; LLM critique merged conservatively; critique failure degrades to rules
test_tools	test_registry.py	register + call; unknown → ToolNotFoundError; disallowed permission rejected; default registry has builtins, no network tool; calculator rejects names/calls; div-by-zero handled; deviation math; arg validation
	test_fs_reader.py	reads within root; ../ traversal blocked ; missing file error
test_api	test_endpoints.py	/health; full task lifecycle (POST → GET → /pipeline → /audit); 404; 422 on empty task
test_integration	test_end_to_end.py	offline end-to-end (all nodes complete, verdict present, required audit events); sovereign records policy_enforced and stays local ; confidential forces local

28. Integration guide (per person P2–P6)

The orchestrator (P1) owns the **brain** — planning, the DAG, routing, execution, the audit stream. It does **not** own your domain service. You expose a small, typed surface; P1 calls it. Below is exactly where and how.

General wiring points

There are three seams P2–P5 will use:

Seam	Where	Use it for
Tool	subclass <code>BaseTool</code> , add to <code>src/tools/builtin/__init__.py</code> <code>BUILTIN_TOOLS</code> , grant the permission in <code>default_registry()</code>	anything the agent should <i>call</i> with explicit arguments (KB search, structured-evidence fetch, DB lookups, deterministic calculations)
MemoryStore	implement <code>src/memory/base.py:MemoryStore</code>	vector store / persistent KB index that outlives a single run
Audit sink	<code>AuditTrail(on_event=<fn>)</code>	stream every event to a file / DB / UI socket

The agent calls a tool by emitting `{"tool": "<name>", "arguments": {...}}`; your tool returns a Pydantic `OutputModel`; the framework wraps it in `ToolResult` and logs + audits it. **Return inputs + formula + output** for anything numeric so P5 can re-verify.

P2 — RAG / Knowledge Base

What P1 will call: two tools (register both in `BUILTIN_TOOLS`).

```
from typing import Dict, List, Optional, Union
from pydantic import BaseModel, Field, validator
from src.memory import MemoryStore
from src.tools.base import BaseTool
from src.tools.builtin import BUILTIN_TOOLS
from src.registry import default_registry

class KBSearch(BaseTool):
    """Search the knowledge base for relevant information based on a query.
    Returns a list of relevant documents or None if no results are found.
    """
    query: str = Field(..., description="The search query")
    limit: int = Field(10, description="Maximum number of results to return")

    def __init__(self, memory_store: MemoryStore):
        self.memory_store = memory_store

    def call(self, arguments: Dict[str, Union[str, int]]) -> List[Dict[str, str]]:
        """Execute the search tool by querying the knowledge base.
        Returns a list of relevant documents or None if no results are found.
        """
        query = arguments.get("query", "")
        limit = arguments.get("limit", 10)

        results = self.memory_store.search(query, limit)

        return results

class FormulaCalculator(BaseTool):
    """Calculate the result of a mathematical formula.
    Returns the result of the calculation or None if the formula is invalid.
    """
    formula: str = Field(..., description="The mathematical formula to calculate")

    def __init__(self):
        pass

    def call(self, arguments: Dict[str, Union[str, int]]) -> Union[int, float]:
        """Execute the formula calculator tool by evaluating the formula.
        Returns the result of the calculation or None if the formula is invalid.
        """
        formula = arguments.get("formula", "")

        try:
            result = eval(formula)
        except Exception:
            return None

        return result
```

Then:

```
# src/tools/builtin/__init__.py
from .kb import KnowledgeBaseSearchTool, ExtractStructuredEvidenceTool
BUILTIN_TOOLS = [..., KnowledgeBaseSearchTool, ExtractStructuredEvidenceTool]
```

Give the researcher / analyst roles access: add the names to `_DEFAULT_TOOLS` in `src/agents/roles.py` (or let the planner put them in a step's tools list).

Persistent index: if you want the KB index to live across executions, implement `MemoryStore` (vector-backed) and pass it where P1 constructs the orchestrator. Interface: `get/set/delete/keys/get_many`.

Contract expectations (from the roadmap): every evidence item must carry `text`, `source`, `page`, `score` and preferably `document_id`. Structured facts must be usable directly as tool arguments to P4's deterministic functions — no number scraping.

P3 — Document Intelligence / OCR / Vision

P3 feeds P2, not P1 directly, but if you want the agent to trigger document processing as a step, expose it as a tool:

```
class ProcessDocumentTool(BaseTool):
    name = "process_document"
    description = "Parse a document (text/OCR/tables) into pages with provenance."
    permissions = (ToolPermission.READ_FILESYSTEM,) # reads from data/
    InputModel = _In # {relative_path: str}
    OutputModel = _Out # DocumentResult: pages[] with text/tables/images + document metadata + source/page ids
    async def _run(self, args): ...
```

Return a **normalised `DocumentResult`** (not a plain string): pages with `text`, `tables` (machine-readable), optional `images`, plus `document_id` and per-page page numbers preserved through every transformation. That structure is what P2's ingestion consumes and what page-level citations depend on.

If OCR/parse is heavy, run it out-of-band (P2 ingestion pipeline) and keep P1's tool thin.

P4 — Tools / Database / Deterministic Computation

This is the most natural fit for the tool system. You already have working examples to copy:

`src/tools/builtin/calculator.py` (`DeviationTool`).

```
# src/tools/builtin/equipment.py (P4 owns it)
class GetEquipmentInfoTool(BaseTool):
    name = "get_equipment_info"
    description = "Structured equipment record for an equipment id."
    permissions = (ToolPermission.PURE,) # local SQLite
    InputModel = _In # {equipment_id: str}
    OutputModel = _Out # dict/BaseModel: tag, service, design limits, ...
    async def _run(self, args): ...

class GetEquipmentHistoryTool(BaseTool):
    name = "get_equipment_history"
    ... # list[MaintenanceRecord]

class CompareParameterTool(BaseTool):
    name = "compare_parameter"
    ... # deterministic; return {inputs, formula, output, units, status}

class CalculateRiskScoreTool(BaseTool):
    name = "calculate_risk_score"
    ... # deterministic; return {inputs, formula, output, status}
```

Rules (framework-enforced or roadmap-required):

- **No agent/planning logic inside tools.** A tool takes explicit arguments and returns a value. The agent decides *when* to call it.
- **Return `inputs`, `formula`, `output`, `units`, `status`** for every calculation so P5 can independently recompute it. `DeviationTool` already models this — mirror its `OutputModel`.
- Declare an `InputModel` (required) and ideally an `OutputModel`. Bad arguments become `ToolResult(ok=False, error="invalid arguments: ...")` automatically — you don't validate manually.
- Permission: local SQLite = PURE. Only use NETWORK if you truly reach off-box, and then P1 must build the registry with `default_registry(allow_network=True)` for a deployment that allows it.

- Register in `BUILTIN_TOOLS`; add to role tool lists in `roles.py` (analyst / executor).

Every tool call is automatically: validated → timed → logged (`tool_call`) → audited (`TOOL_CALLED` / `TOOL_FAILED` with sanitised input) → attached to the node's `AgentResult.tool_results` and to `PipelineState.tool_results`.

P5 — Verification, Audit & Security

Verification. P1 already runs `ResultVerifier` as the terminal `verify_final` node and after finalisation. Two ways to extend:

1. **Strengthen the rule layer** — edit `src/verification/verifier.py`: add issue checks (claim-to-evidence linkage, numeric re-computation from `ToolResult` data, contradiction detection across evidence). Keep returning `VerificationResult`.
2. **Provide your own verifier** — build a class with the same `async def verify(task, result, *, require_evidence=False, require_json=False, structured=None) -> VerificationResult` signature and pass it into `PipelineExecutor(verifier=<yours>)` (and/or where the orchestrator constructs `ResultVerifier`). The executor and orchestrator only depend on that method + the `VerificationResult` shape.

What you get to work with: the full run state — `ExecutionContext.state.node_results` (each `AgentResult` with its `tool_results`, i.e. every calculation's inputs/formula/output), `ctx.state.verifications`, `ctx.plan`, `ctx.pipeline`. Numeric verification = re-run P4's deterministic function with the recorded inputs and compare.

Contract rules to preserve: confidence must reflect evidence quality (not a decorative number); never convert "no evidence" into a pass; flag conflicts rather than silently choosing; recommendation drives `retry` / `correct` / `replan` / `escalate` / `reject`.

Audit / security panel. Everything is already event-sourced. To build the security-posture panel and a durable audit log:

```
def sink(event: AuditEvent) -> None:
    db.insert(event.model_dump(mode="json"))    # or append JSONL, or push to a UI socket

audit = AuditTrail(on_event=sink)
orchestrator = Orchestrator(settings, audit=audit)
```

`GET /tasks/{id}/audit` already returns the ordered, redacted event list. The security-posture values you need (local LLM, local everything, external calls = 0) are derivable: `settings.sovereign_mode`, each node's provider + `is_local`, and the absence of `NETWORK`-permission tools in the registry (`registry.specs()`).

P6 — UI, Human Approval & Integration

You do not need to parse logs. Everything the UI needs is JSON from four endpoints:

UI element	Source
Document upload / inventory	your own storage + P3 <code>process_document</code>
Task input + "Run Agent"	<code>POST /tasks {"task": "...", "confidential": <bool>}</code>
Plan viewer ("what the agent intends to do")	<code>GET /tasks/{id} → ...</code> or <code>GET /tasks/{id}/pipeline → nodes[].{name,type,description,depends_on}</code> (also <code>ctx.plan</code> in <code>summary()</code>)
Live execution trace	<code>GET /tasks/{id}/audit → ordered events[]</code> (poll while status is running)
Pipeline graph (nodes, edges, status colours)	<code>GET /tasks/{id}/pipeline → nodes[].status ∈ pending/running/completed/failed/skipped/retrying, edges[]</code>
Per-node provider / model / tools / timing	<code>nodes[].{provider,model,tools,duration_ms,attempts}</code>
Intermediate results	<code>nodes[].result</code> (full <code>AgentResult</code> , incl. <code>tool_results[]</code> with each tool's input/output)

UI element	Source
Findings / evidence viewer	<code>nodes[].result.tool_results[]</code> where <code>tool</code> is <code>search_knowledge_base / extract_structured_evidence</code> → <code>output.evidence[]</code> with <code>source</code> , <code>page</code>
Verification panel (verified / unverified / conflict)	<code>GET /tasks/{id} → final_verification.{passed,score,recommendation,issues[]};</code> per-node in <code>summary().verifications</code>
Errors / retries	<code>nodes[].error</code> , <code>nodes[].attempts</code> ; <code>execution.error</code>
Final report (after approval)	<code>execution.final_answer</code> — gate finalisation in your state machine

Human approval state machine (**DRAFT** → **PENDING_REVIEW** → **APPROVED** / **REVISION_REQUESTED** / **REJECTED**) lives in the UI/service layer. The backend produces a **draft** answer + verification; it does not currently block on approval. Recommended integration:

- Treat `execution.status == "completed"` with `final_verification.passed == true` as **ready for PENDING_REVIEW**.
- `status == "needs_review"` (a node failed or the verdict didn't pass) → surface `final_verification.issues` and the failing node, route straight to **REVISION_REQUESTED** guidance.
- Only expose "Download report" once your state reaches **APPROVED**.

Polling model: `POST /tasks` runs synchronously today (returns when the run is done). For a live trace during long local-model runs, the current shape still works (the response carries the finished result and you replay the audit trail); a streaming/async job variant is a listed follow-up (see §29).

29. Known limitations / not yet implemented

These are genuine gaps, called out so nobody assumes otherwise:

1. **`POST /tasks` is synchronous.** The request blocks until the run finishes. No job queue, no SSE/WebSocket trace stream yet. Fine for the demo and short local-model runs; a background-job variant (`202 Accepted` + poll) is the natural next step.
2. **Verifier replan recommendation is not auto-actioned.** `retry / replan` counters exist and are enforced, but the executor does not yet loop a failed verification back into the planner automatically. `ExecutionStatus.NEEDS_REVIEW` is set instead.
3. **Conditional edges** (`PipelineEdge.condition`) are modelled but the executor does not evaluate predicates — every edge is treated as an unconditional dependency.
4. **Execution store is in-process.** The API keeps `ExecutionContexts` in a dict; restart loses history. `MemoryStore / AuditTrail(on_event=...)` are the seams for persistence (`SQLite/Postgres/vector`) — implementations not included.
5. **`configs/settings.yaml` and `configs/tools.yaml` are documentation-level.** Runtime behaviour is driven by `.env / Settings` and `configs/models.yaml`. Wiring `tools.yaml` into `default_registry()` is a small isolated change.
6. **No auth / RBAC / rate limiting** on the API. Architecturally open; no middleware yet.
7. **Node-level parallelism is asyncio only** (single process). Good for I/O-bound LLM/tool calls; no multiprocessing / distributed execution.
8. **replans counter on `ExecutionContext`** is present but only the planner's internal attempt budget is currently incremented; end-to-end replan loop is item 2.
9. **RAG / OCR / DB tools are not implemented here** — they are P2/P3/P4 deliverables. This backend ships the tool *framework* and example deterministic tools only.
10. **The `openai` Python SDK is intentionally not installed.** Providers use `httpx`. If another workstream imports `openai`, add it back to `requirements.txt` (it is listed as an optional comment).

30. Extension recipes

Add an LLM provider

1. `src/llm/<name>_provider.py`: subclass `BaseLLM`, implement `async def _complete(self, request) -> LLMResponse`, set `provider_name`, `is_local`.
2. `src/llm/factory.py`: add one `if name == "<name>":` branch.
3. `src/core/config.py`: add the value to `LLMProvider` (and to `_LOCAL_PROVIDERS` if it stays on-box).
4. No agent, planner, pipeline, or verifier change.

Add a tool

1. Subclass `BaseTool`, define `name`, `description`, `permissions`, `InputModel` (required), `OutputModel`, `async def _run(self, args)`.
2. Add the class to `src/tools/builtin/__init__.py::BUILTIN_TOOLS`.
3. If it needs a non-PURE permission, ensure `default_registry()` grants it.
4. Add the `name` to the relevant role in `src/agents/roles.py::_DEFAULT_TOOLS`, or let the planner assign it per step.

Add an agent role

1. `src/agents/roles.py`: add a prompt string and an entry in `_PROMPTS` (+ optional `_DEFAULT_TOOLS`).
2. Add the role name to `planner.py::_KNOWN_AGENTS` and mention it in the planner system prompt if the planner should select it.

Persistent memory / vector store

Implement `src/memory/base.py::MemoryStore` (`async get/set/delete/keys`), pass the instance where the orchestrator is constructed.

Durable audit sink

`AuditTrail(on_event=lambda e: write(e.model_dump(mode="json")))` — exceptions in the sink are caught and logged, never break a run.

Custom verifier

Any object with `async def verify(task, result, *, require_evidence=False, require_json=False, structured=None) -> VerificationResult`. Inject via `PipelineExecutor(verifier=...)`.

31. Appendix

31.1 Dependency changes (`requirements.txt`)

Removed from runtime (unused by this backend; kept as optional comments for RAG/OCR/vision): `openai`, `transformers`, `torch`, `accelerate`, `instructor`, `tenacity`, `numpy`, `pandas`.

Runtime deps now: `pydantic`, `pydantic-settings`, `python-dotenv`, `PyYAML`, `fastapi`, `uvicorn[standard]`, `httpx`, `structlog`, `pytest`, `pytest-asyncio`, `pytest-cov`, `ruff`, `mypy`.

New: `pyproject.toml` — `[tool.pytest.ini_options] asyncio_mode = "auto", testpaths = ["tests"]`; `ruff config (E,F,I,UP,B,C4, line-length 100)`; `mypy config (ignore_missing_imports)`.

31.2 Full new-file inventory (40 source modules)

```
src/orchestrator.py
src/llm/ollama_provider.py  src/llm/vllm_provider.py  src/llm/echo_provider.py
src/agents/planner.py      src/agents/roles.py
src/pipeline/__init__.py   src/pipeline/models.py      src/pipeline/graph.py
src/pipeline/builder.py    src/pipeline/executor.py    src/pipeline/state.py
src/tools/builtin/__init__.py src/tools/builtin/calculator.py
src/tools/builtin/text_stats.py src/tools/builtin/json_tool.py src/tools/builtin/fs_reader.py
src/verification/schemas.py src/verification/verifier.py
src/memory/base.py         src/memory/in_memory.py
src/audit/events.py        src/audit/trail.py
src/api/schemas.py
.env.example                pyproject.toml
tests/conftest.py
tests/test_core/...  tests/test_pipeline/...  tests/test_api/...  (+ new files under existing test dirs)
```

Modified in place (backward compatible): src/core/{config, logging, exceptions}.py,
src/llm/{base, schemas, factory, openai_provider, local_provider, __init__}.py,
src/agents/{base_agent, inspection_agent, router, __init__}.py, src/tools/{base_tool, registry, __init__}.py,
src/memory/{memory_manager, __init__}.py,
src/{main, api/app, api/__init__, prompts/system_prompts, prompts/templates}.py,
src/{audit, verification, state}/__init__.py, configs/*.yaml, .gitignore, README.md, requirements.txt.

31.3 Glossary

Term	Meaning
Sovereign mode	SOVEREIGN_MODE=true; cloud providers are impossible to select; all inference stays on the machine; enforced at config load and routing time; audited
Confidential task	per-request confidential=true; forces local-only routing for that one execution even when sovereign mode is off
Node	one step of the pipeline DAG, executed by one agent role
Layer	a set of nodes with no unmet dependency, run concurrently
Leaf node	a node no other node depends on; the terminal verifier depends on all leaves
AgentResult	everything one agent produced on one node (output text, tool calls, tokens, timing)
ToolResult	one tool invocation's sanitised input, output, ok/error, duration
VerificationResult	verifier verdict: passed, score 0–1, issues[], recommendation
ExecutionContext	the single object carrying one run's task, plan, pipeline, state, answer, verdict, status
Echo provider	deterministic offline stand-in for a model; makes the whole pipeline runnable with zero setup
P1...P6	the six team roles: P1 orchestration (this doc), P2 RAG/KB, P3 document intelligence, P4 tools/DB, P5 verification/governance, P6 UI/HITL

End of document. Questions about the orchestration layer → P1. This document tracks commit 743677e on feature/agent-orchestration-backend.